Mid-Sem Report

Atomic Fee Enforcement

in Decentralized Perpetual Trading

A Transaction-Layer Approach Using Elliptic Curve Cryptography

Course: BLOCKCHAIN TECHNOLOGY (BITS F452)

Instructor In-Charge: Prof. Sanjay K. Sahay

Name of the Student	ID.No.
Anshul Shah	2022B3A70406G
Viyom Gupta	2023A7PS0413G

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(March, 2026)

Contents

1 Introduction **2**
1.1 Background 2
1.2 Problem Statement 2
1.3 Proposed Solution 2
1.4 Objectives 2
1.5 Technology Stack 3

2 Literature Survey **3**
2.1 Solana Transaction Model 3
2.2 Cryptographic Foundations 3
2.3 Drift Protocol 3
2.4 Related Work 4

3 Plan of Action **4**
3.1 Three-Phase Methodology 4
3.2 Project Timeline 4

4 Work Completed Till Mar. 2026 **4**
4.1 Wallet Interaction Layer 4
4.2 Blockchain Interaction Layer 5
4.3 Transaction Construction Layer 5
4.4 Fee Computation Module 5
4.5 Frontend Interface 5

5 End Semester Deliverables (Work Yet to be Completed) **6**
5.1 Data Persistence Layer 6

6 References **6**

1 Introduction

1.1 Background

Decentralized Exchanges (DEXs) on the Solana blockchain have become critical DeFi infrastructure, processing billions in trading volume. Solana's architecture — sub-second block times, low transaction fees (~$0.00025), and high throughput — makes it ideal for performance-sensitive trading applications. However, a fundamental vulnerability exists in how these platforms collect fees.

1.2 Problem Statement

Modern DEXs separate trade execution and fee collection into independent blockchain transactions. This separation creates several critical issues:

- **Fee Evasion**: Users can execute trades while bypassing fee payment transactions.
- **Partial Execution**: Network congestion can cause fee transactions to fail while trades succeed.
- **Race Conditions**: Malicious actors can exploit timing gaps between fee and trade transactions.

This breaks the principle of atomic execution — that related operations should succeed or fail together — undermining DEX sustainability and trustless execution.

1.3 Proposed Solution

Atomic Fee Enforcement binds fee payment and trade execution into a single Solana transaction. Solana's native transaction model supports multiple instructions with atomic guarantees — all succeed or all fail. By prepending a fee transfer instruction to the trade transaction and signing both under a single Ed25519 signature, we achieve cryptographic indivisibility.

1.4 Objectives

1. **Atomic Fee Bundling** — Bind fee and trade into a single all-or-nothing transaction.
2. **Ed25519 Signing Integration** — Single signature authorizes both operations.
3. **SHA-256 Integrity** — Prevent post-signature tampering of fee/trade parameters.
4. **Trade Primitive Support** — Cover market, limit, and stop-loss order types.
5. **Solana Versioned Tx Architecture** — Leverage V0 transactions and Address Lookup Tables.
6. **Attack Resistance** — Validate against replay, forgery, and bypass attacks.

1.5 Technology Stack

Component	Technology
Frontend	Next.js 15, React 19, TypeScript
Blockchain	Solana web3.js 1.98
DeFi Protocol	Drift SDK 2.143
Wallet	Phantom (Solana Wallet Adapter)
State Management	Zustand 5
Database	MongoDB (Mongoose 8)

2 Literature Survey

2.1 Solana Transaction Model

Solana transactions contain a message with one or more instructions that execute atomically — if any instruction fails, the entire transaction reverts. Versioned Transactions (V0) support Address Lookup Tables for more complex instruction sets. Transactions are signed using Ed25519 over the serialized message, ensuring all instructions are covered by a single cryptographic proof.

2.2 Cryptographic Foundations

Ed25519 (RFC 8032) provides 128-bit security, deterministic signing, and 64-byte signatures. A single Ed25519 signature over the transaction message simultaneously authorizes all instructions within the transaction — this is the key property enabling atomic fee enforcement.

SHA-256 (FIPS 180-4) is used in Solana's signing pipeline to hash the transaction message before signing. Any modification to any instruction changes the hash and invalidates the signature, providing tamper detection.

2.3 Drift Protocol

Drift Protocol is Solana's largest decentralized perpetual futures exchange, supporting 50+ markets with up to 20x leverage. The AuthorityDrift SDK provides transaction construction methods (`sendTransaction`, `openPerpOrder`) that serve as natural injection points for fee instruction prepending.

2.4 Related Work

Platform	Fee Mechanism	Atomicity
Jupiter	Fees inside swap program	Program-level
Raydium	LP fees in AMM math	Program-level
Drift	Builder fees via RevenueShare	Protocol-level
Our Approach	**Instruction prepending**	**Transaction-level**

Our approach operates at the application layer without requiring on-chain program modifications, making it portable across any DEX protocol.

3 Plan of Action

3.1 Three-Phase Methodology

1. **Phase 1: Research & Design** (Jan) — Study Solana transaction model, Drift SDK, and signing workflows. Design fee interception architecture.
2. **Phase 2: Development** (Feb–Mar) — Implement wallet integration, transaction construction, fee computation, frontend interface, and devnet testing.
3. **Phase 3: Validation** (Mar–May) — Complete atomic binding, attack resistance testing, performance benchmarks, and SHA-256 integrity verification.

3.2 Project Timeline

Phase	Period	Activities	Status
Research & Design	Jan W3	Solana tx model, architecture	Completed
Wallet & Devnet	Feb W1	Phantom, devnet setup	Completed
Tx Construction	Feb W2	Multi-instruction tx building	Completed
Fee Computation	Feb W4	Fee calculator, oracle prices	Completed
Frontend	Mar W1	Trading UI, charts, orderbook	Completed
Atomic Binding	–	Fee + trade bundling	In Progress
Attack Validation	–	Adversarial testing	Planned
Benchmarks	–	Overhead measurement	Planned

4 Work Completed Till Mar. 2026

4.1 Wallet Interaction Layer

Built using `@solana/wallet-adapter-react` with Phantom as the primary wallet. Supports wallet connection, public key extraction, and Ed25519 transaction signing — the cryptographic foundation for atomic authorization. Wallet sessions persist across page reloads with auto-reconnect.

Verified: Phantom connected on Solana devnet, balance updates confirmed after SOL airdrop via faucet.

4.2 Blockchain Interaction Layer

Dual-environment configuration supporting seamless switching between Solana devnet and mainnet-beta, with environment preference persisted via Zustand store. SOL airdrop via Solana faucet enables risk-free testing. Real-time oracle price subscriptions from Pyth and Switchboard provide live market data. All transactions verified on Solscan devnet explorer with full instruction details visible.

Verified: Transactions submitted, confirmed, and inspected on Solscan showing instruction details.

4.3 Transaction Construction Layer

Supports building multi-instruction transactions for legacy `Transaction` & `VersionedTransaction` (V0) types. Handles Address Lookup Table resolution, transaction message decompilation and recompilation via `TransactionMessage.decompile()` and `compileToV0Message()`, and `SystemProgram.transfer` instruction creation for fee payments. This is the core capability required for prepending fee instructions to trade transactions.

Verified: Multi-instruction transactions successfully signed, submitted, and confirmed on devnet.

4.4 Fee Computation Module

Calculates platform fees at 5 basis points (0.05%) of order size. For base assets (e.g., SOL), fees are computed directly in lamports. For quote assets (e.g., USDC), fees are converted to SOL using live oracle prices from Pyth/Switchboard. Outputs a `SystemProgram.transfer` instruction with a configurable recipient address via the `NEXT_PUBLIC_BUILDER_AUTHORITY` environment variable.

Verified: Fee calculations validated for various order sizes across both asset types.

4.5 Frontend Interface

Comprehensive trading interface built with Next.js 15 and React 19:

- 40+ perpetual markets with searchable selector and market icons
- Candlestick charts with real-time WebSocket price streaming
- Trade form supporting 5 order types: market, limit, take profit, stop loss, oracle limit
- Orderbook display, positions table, and open orders table

- Leverage slider from 1x to 20x with dynamic position sizing

Five Zustand stores provide reactive state management for Drift client, oracle prices, mark prices, user accounts, and user identity.

Verified: Full UI accessible at `localhost:3000/perps` with real-time price updates.

5 End Semester Deliverables (Work Yet to be Completed)

Task	Description	Timeline
Atomic Binding	Complete cryptographic binding of fee + trade into a single signed transaction	Mar
SHA-256 Verification	Validate that post-signature modifications invalidate the transaction	Apr
Attack Testing	Replay attacks, signature forgery, fee bypass, instruction reordering	Apr
Benchmarks	Transaction size overhead, signing latency, compute unit consumption	Apr
Revenue Share	50/50 platform-creator fee split with claim workflow	May
Data Persistence	MongoDB integration for transaction audit trail (Fee and Trade schemas)	May

5.1 Data Persistence Layer

MongoDB integration with Mongoose ODM will provide a transaction audit trail. Two schemas to be implemented:

- **Fee Schema**: userId, experienceId, feeAmount, feeInLamports, orderSize, assetType, timestamp
- **Trade Schema**: userId, marketIndex, orderType, direction, size, subAccountId, txSignature, timestamp

API endpoints (`POST/GET /api/fee`, `POST/GET /api/trade`) will support recording and querying transaction history with pagination.

6 References

1. Y. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain," Solana Labs Whitepaper, 2018.
2. Solana Foundation, "Transaction Processing," *Solana Documentation*, 2024. Available: `https://docs.solana.com/developing/programming-model/transactions`

3. S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, 2017.
4. National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS 180-4, 2015.
5. Drift Labs, "Drift Protocol Documentation," 2024. Available: `https://docs.drift.trade/`
6. Drift Labs, "Drift Protocol v2 — GitHub Repository," 2024. Available: `https://github.com/drift-labs/protocol-v2`
7. A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
8. D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, 2020. Available: `https://toc.cryptobook.us/`
9. Phantom, "Developer Documentation," 2024. Available: `https://docs.phantom.app/`
10. Solana Foundation, "Solana Wallet Adapter," GitHub, 2024. Available: `https://github.com/solana-labs/wallet-adapter`